B.TECH PROJECT · COMPUTER SCIENCE AND ENGINEERING

Tessera

A privacy-preserving, hardware-independent local inference and screen-aware memory system for Windows



Pranshu Ranjan · BTECH/15191/24 · Guide: Dr. Ayan Kumar Das

Birla Institute of Technology, Mesra — Off-Campus Patna

Stages 0–7 · v0.0.1 · research prototype under evaluation, not a released product

Useful context is private context

An assistant that knows what you are working on is only useful because it can see what you are working on: your mail, your chats, your documents. Almost every product that does this sends that text to a remote server, because the strongest models are too large to run on a laptop.

Running the model locally removes that exposure — and replaces it with four engineering problems that the cloud design hides.

Who is affected: anyone whose screen holds work they cannot send to a third party — students, clinicians, lawyers, engineers under NDA.

1

Hardware fragmentation

CUDA, ROCm, SYCL, Metal and OpenCL are mutually incompatible. Ship one and you serve one vendor; ship all and the CUDA runtime alone is ~390 MB.

2

Process fragility

A 2–6 GB model pushes consumer GPUs to their limit. In-process inference means a driver reset takes the application, and possibly the database, with it.

3

Exposure of captured text

A system that reads the screen will eventually read a password manager, a banking page or an identity number. By the time the model sees it, the exposure has happened.

4

Misplaced authority

A web page can contain a sentence addressed to the model, not the user. A model cannot reliably tell its operator's instructions from text it was asked to read.

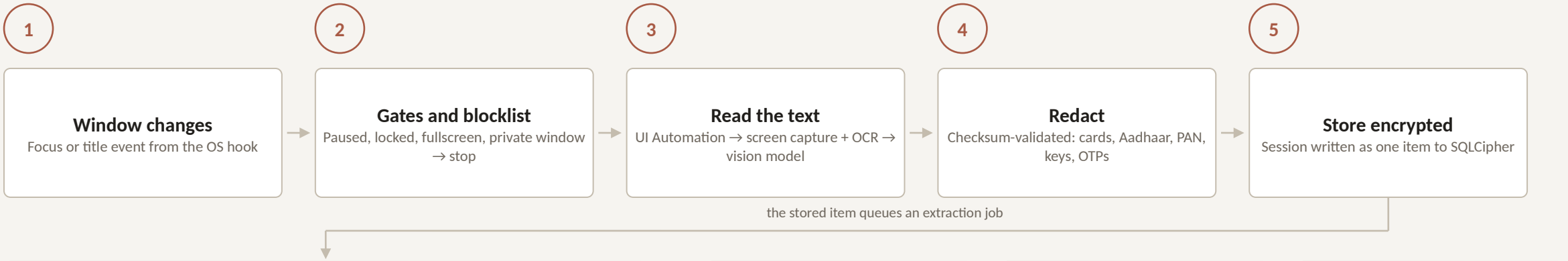
The model proposes. Code disposes

No guarantee offered to the user may depend on the language model behaving correctly. Wherever the model produces something, deterministic code checks it before it has any effect.

	CONVENTIONAL ASSISTANT	TESSERA
Where the text goes	To a provider's servers	Never leaves the machine; inference on 127.0.0.1
What the model is trusted with	Reading, judging and often acting	Proposing JSON only — it has no tools and cannot act
What you can check afterwards	The answer, and whether it sounds right	Every fact carries the verbatim span it came from
What an injected instruction achieves	Whatever the model's tools allow	At most a misleading line of text you read

The right-hand column is enforced by code paths and database constraints, not by prompt wording — which is why slide 12 can report a hallucination rate of zero while the model's accuracy misses its target.

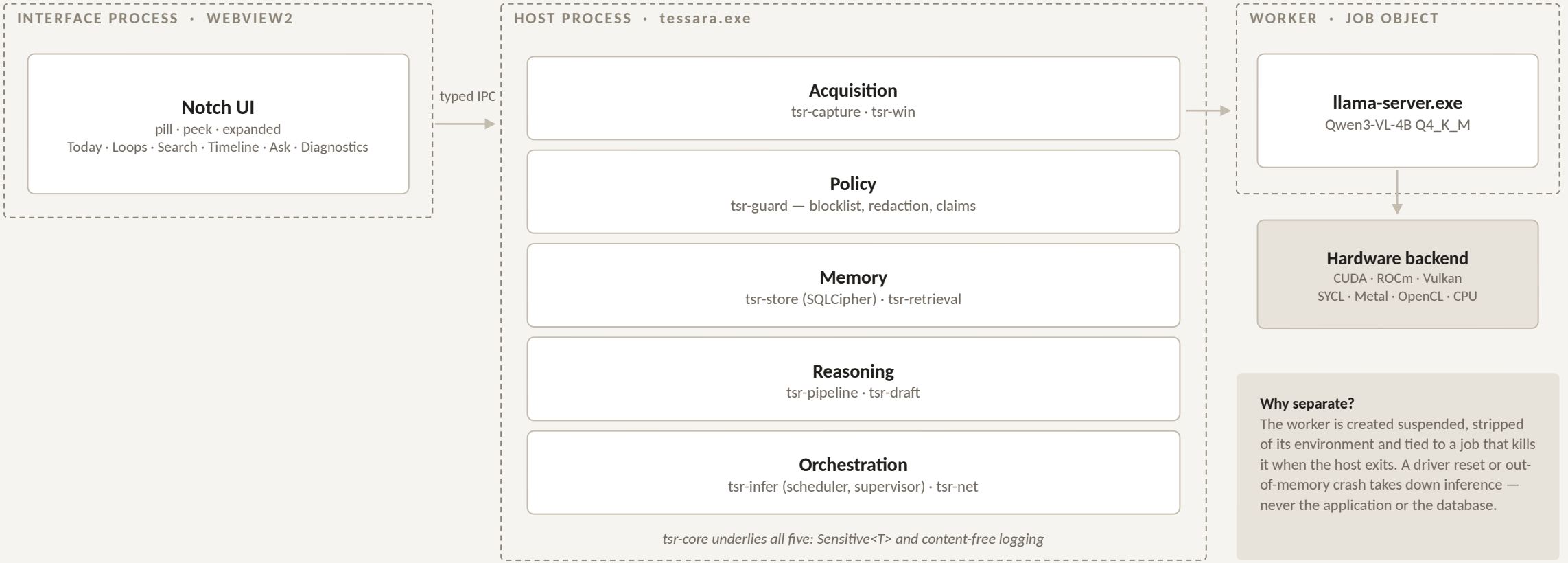
From a window changing to an answer you can check



- **Capture starts paused** Nothing is read until you switch it on; pausing discards buffered text.
- **No screen image is written** Frames exist in memory only.
- **Redaction runs before storage** And again on the model's own output.

ARCHITECTURE AT A GLANCE

Three processes, one of which is allowed to fail



Why separate?
The worker is created suspended, stripped of its environment and tied to a job that kills it when the host exits. A driver reset or out-of-memory crash takes down inference — never the application or the database.

Dashed outlines are operating-system process boundaries. The host reaches the worker only over 127.0.0.1, and only after confirming that the process owning the listening socket is the child it created. It trusts neither of the other two: everything from the interface is validated, everything from the worker is an unverified proposal.

Boundaries drawn where the risk is, not where it is convenient

`tsr-capture` 5,101 lines
Three-rung acquisition ladder, the 285 ms deadline worker, SimHash change detection, session aggregation.

`tsr-guard` 1,651 lines
Blocklist, redaction engine, claim checker. Deliberately platform-neutral.

`tsr-store` 3,081 lines
SQLCipher database, forward-only migrations, full-text indexes, repositories.

`tsr-retrieval` 1,621 lines
Pinned ONNX runtime, chunking, int8 vectors, hybrid search, the Ask pipeline.

`tsr-infer` 8,664 lines
Adapter discovery, GGUF parsing, backend selection, supervisor, loopback client, scheduler.

`tsr-pipeline` 7,254 lines
Extraction, schema, evidence verification, injection guard, resolvers, rollups.

`tsr-draft` 587 lines
Draft generation and the independent fact checker.

`tsr-net` 1,115 lines
The only network-capable crate. Approval-gated, digest-verified downloads.

`tsr-win` 1,771 lines DPAPI, job objects, display affinity, window hooks, clipboard, TCP table.

`tsr-core` 475 lines Sensitive<T>, content-free logging allowlist.

- **Security code compiles without Windows**
  tsr-guard, tsr-store, tsr-pipeline and tsr-retrieval have no Windows dependency, so the 355-case secrets corpus and the 152-case injection corpus run in ordinary CI.
- **Network capability is a single crate**
  A build-time checker walks the whole dependency graph and fails compilation if any other crate can reach a network-capable dependency. It is itself tested against broken fixtures.
- **One crate per failure domain**
  Acquisition can time out, inference can crash, downloads can be refused — each is contained by the boundary it sits behind rather than by discipline.

TECHNOLOGY STACK

Every layer chosen for a constraint, not a preference

Interface	`React 19 · TypeScript · Vite · Tailwind · Tauri 2.12.5`	Tauri hosts the UI in the system WebView2 runtime instead of bundling a browser engine, which keeps the binary small. Trade-off: WebView2 is also the one component whose network behaviour the project does not control.
Host and orchestration	`Rust 2024 (1.99+) · Cargo workspace, 10 crates`	Memory and thread safety matter because the capture path handles untrusted text on background threads. The workspace boundary is what makes the network check in slide 6 enforceable at compile time.
Operating system	`UI Automation · Windows Graphics Capture · WinRT OCR · DPAPI · Job Objects · display affinity`	There is no single way to read another application's text on Windows, so all three mechanisms are used in sequence. DPAPI ties the database key to the Windows login; job objects guarantee the worker dies with the host.
Storage and retrieval	`SQLCipher via rusqlite 0.41.0 · SQLite FTS5 · fastembed 7.5.0 on ONNX Runtime 1.31`	Whole-database AES-256 encryption including indexes and the write-ahead log. Default features of the embedding crates are switched off because they would pull in an HTTP client and a build-time download.
Inference	`llama.cpp b11900 · Qwen3-VL-4B-Instruct Q4_K_M · multilingual-e5-small`	The engine is an external, hash-verified binary rather than a linked library — the whole basis of the hardware independence on the next slide. Quantisation brings a 4B vision-language model to about 2.4 GB.

Versions are read from Cargo.toml. Security-sensitive dependencies are pinned to exact patch versions, not compatible ranges.

Three decisions that define the system

1 Hardware independence

PROBLEM

GPU vendors ship incompatible runtimes. Linking one serves one vendor; linking all is unshippable.

APPROACH

Backends live outside the binary as separately pinned builds. At run time: enumerate adapters without loading any vendor runtime, hash-verify every file of each build against a compiled-in manifest, probe each build for real with a device-listing flag, read the model's own GGUF header, then rank candidates with the CPU always last.

RESULT

No vendor library is in the dependency graph; the application builds and runs with no GPU SDK installed. The model is never re-quantised to fit — an unsupported pairing is skipped.

Two backend families verified on real hardware; five on simulated adapter reports only.

2 The 285 ms deadline

PROBLEM

Accessibility reads are synchronous calls into another process. Microsoft Word blocked for 2.8 seconds in measurement.

APPROACH

Reads run on a dedicated apartment-threaded worker. The caller waits 285 ms and then proceeds without a result; the abandoned read finishes in the background and its answer is discarded.

RESULT

Across 16 windows and 9 applications the capture loop never blocked longer than 294 ms, while the abandoned reads themselves ran on for up to 732 ms.

Word, PowerPoint and File Explorer hit the deadline every time and are served by OCR instead.

3 Evidence, or nothing

PROBLEM

A model asked to summarise will produce fluent statements the source does not support — and a memory system is believed later.

APPROACH

Every proposed fact must carry a quotation. Code looks for it: exact substring, then after normalisation, then a fuzzy word-set match at Dice ≥ 0.9. A fact whose quote is absent is discarded, never repaired. The schema requires at least one evidence row and a trigger deletes a fact when its last one goes.

RESULT

Of 97 facts stored in the real-model run, zero carried evidence absent from the source, and zero rows violated the schema.

Provenance is guaranteed; completeness is not — recall is the weak half (slide 12).

A fourth decision underlies all three: the scheduler preempts background extraction by dropping its HTTP stream, so an interactive request never waits behind a summary.

What is enforced, what is merely present, and what is out of reach

- **Interface → host**
  Fixed command set; lengths, enumerations and identifiers validated; status changes need a single-use approval token; errors return a code, never content.

- **Host → worker**
  Loopback only. The listening socket's owning process ID must equal the spawned child's, so no other local process can impersonate the worker.

- **Application → disk**
  Whole-database AES-256. The key is unwrapped from the Windows login, used, then wiped. A wrong key is rejected and writes nothing to stderr.

- **Application → internet**
  All network code in one crate, gated on a typed approval value and a pinned SHA-256 digest. Enforced at build time across the dependency graph.

- **Screen text → prompt**
  Escaped and delimited, and the model has no tools. Its output is grammar-constrained, then re-validated, re-redacted and classified in code.

VERIFIED BY MEASUREMENT

355 / 355 seeded secrets removed, 0.41 % false positives
0 plaintext hits on disk in a 60-second canary scan
0 hallucinated stored facts, 0 fabricated citations
Insertion into a password field or a changed target refused
Interface absent from two independent capture paths

PRESENT BUT NOT ADEQUATELY TESTED

24-hour network and canary runs — only 10 and 60 seconds run
Password exclusion in native WinForms / WPF applications
Clipboard-history and cloud-clipboard exclusion
Coverage-guided fuzzing — randomised testing only

KNOWN GAPS AND OUT OF SCOPE

WebView2 opens one outbound HTTPS connection per run
12 of 39 unseen injections still stored as readable to-dos
Page file and hibernation file not scanned (needs elevation)
An attacker holding the Windows account defeats the design

Tessera is not claimed to be fully sandboxed, air-gapped or immune to attack. The architecture bounds what a successful injection can do — no case in a 152-attack corpus caused an action, revealed a secret or wrote an unexpected table — but it does not prevent one.

Build it, approve a model, then switch capture on

PREREQUISITES

Windows 11 64-bit · Rust stable MSVC (1.99+, edition 2024) · Visual Studio C++ build tools · Strawberry Perl on PATH (the vendored OpenSSL build needs it) · Node.js 24+ with pnpm · Python 3.11+

```
# interface bundle
pnpm install && pnpm build:ui

# pinned, hash-checked inference runtimes
python tools/fetch_sidecar.py llama_cpp cpu-x64
python tools/fetch_sidecar.py llama_cpp vulkan-x64

# verify the boundaries, then build
python tools/check_network_deps.py --locked
cargo test --workspace -- --include-ignored
cargo build -p tessara --features custom-protocol
```

Commands as given in README.md §18.2. The package is named tessara internally while the repository is Tessera — the build command uses the internal name.

1. **Approve a model**
   Nothing is downloaded automatically. The download API needs a typed approval value that only an explicit user action can construct, and the file is accepted only if its SHA-256 matches tools/models.lock.

2. **Switch capture on**
   Capture starts paused on first run. Until you unpause it, nothing is read at all.

3. **Work normally**
   The notch sits at the top of the screen as a pill. Ctrl+Alt+Space opens it; if another application already owns that chord it falls back to Ctrl+Alt+N and the interface tells you which is active.

4. **Ask, or draft**
   Ctrl+Alt+K opens Ask; each answer sentence carries a citation chip that opens the underlying item. For a draft, put the cursor in the reply field first — insertion re-checks the target and never presses Enter.

5. **Pause, or forget**
   The hotkey or tray toggle halts capture instantly and drops buffered text. A person can be forgotten; only a salted digest of the identifier is kept, so the suppression list cannot be read back into names.

REAL-WORLD USE CASES

Three situations the implementation already supports

1 A student across a week of coursework

A submission date agreed in a chat on Monday is gone by Thursday. It was never written anywhere except on screen.

Extraction records it as a commitment with the sentence it came from. It appears in Open Loops and in the morning brief.

`tsr-pipeline · tsr-store`

Needs the window to have been readable and not blocklisted at the time.

2 An engineer returning to an old decision

“Why did we drop the second approach?” — the reasoning is in a thread nobody can find.

Ask retrieves the snippets, the model answers from them, and each sentence shown is verified against the snippet it cites. Unverified sentences are hidden, not shown with a caveat.

`tsr-retrieval · tsr-pipeline`

Correct on 66.7 % of a 60-question set with the 4B model, and slow on integrated graphics.

3 Anyone writing a reply with context

The reply needs a date, an amount and a name that are scattered across the thread and last week's mail.

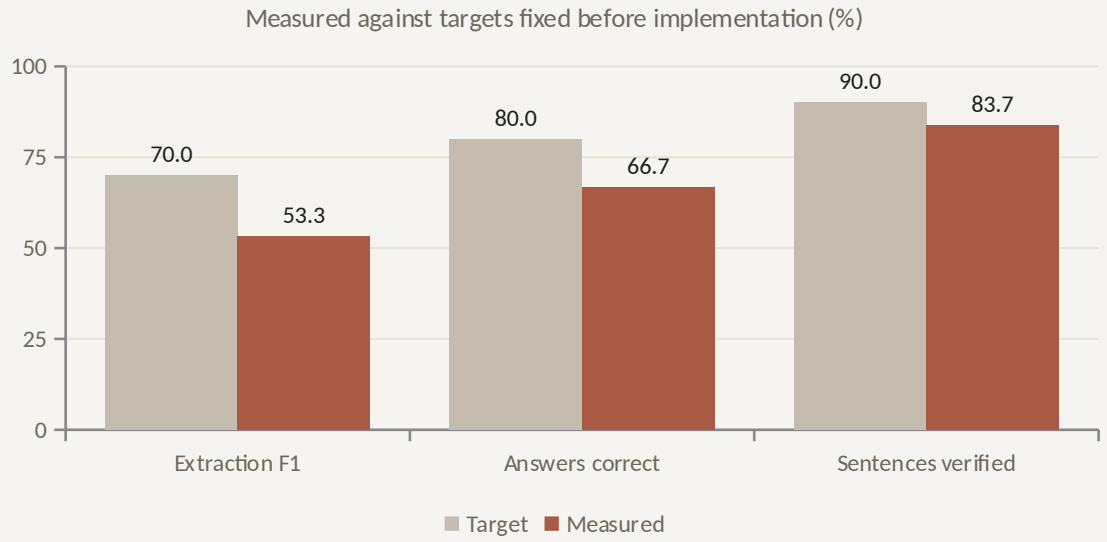
The draft assistant gathers the thread and relevant memory, generates at interactive priority, then an independent checker verifies every date, amount, number and name against the context and flags anything unsupported.

`tsr-draft · tsr-guard`

Verified in a plain text field only; the five named mail and chat applications are untested.

Potential applications not yet built: meeting notes and live translation were specified for a later stage and are not implemented. Nothing here claims the system is pleasant to use at current latency.

Code-enforced guarantees held. Model-dependent ones did not



Qwen3-VL-4B Q4_K_M on an AMD Radeon integrated GPU via Vulkan. The intended reference machine, an RTX 4050, was never available, so every timing is a lower bound.

375 Rust test functions · 355-case secrets corpus · 152-case injection corpus, half of it held out from tuning · 80 extraction items · 60 questions · 20 draft scenarios · 20,000 randomised inputs per robustness target. One criterion — the 8-hour resource and 7-day stability run — has not been run at all.

ELEVEN CRITERIA MET — ALL ENFORCED BY CODE

355 / 355 seeded secrets removed · 0.41 % false positives
0 plaintext hits across 285 files / 32 MB
0 / 97 stored facts hallucinated · 0 schema violations
0 fabricated citations
0 commitments closed without confirmation
Frame p95 16.8 ms · input-to-visual p95 64 ms
100 % absent from Graphics Capture and GDI frames

SEVEN NOT MET — MODEL, HARDWARE OR THIRD PARTY

Extraction F1 0.533 against 0.70
Answers correct 66.7 % against 80 %
Sentences verified 83.7 % against 90 %
First answer sentence 25.2 s against 2.5 s
Complete draft 16.4 s against 8 s
WebView2: one outbound connection per run
Injection: 24 / 39 on the untouched corpus half

What is wrong, why, and what would fix it

LIMITATION	WHY IT EXISTS	PRACTICAL IMPACT	POSSIBLE IMPROVEMENT
Extraction misses too much and over-produces trivia	The 4B model generates many low-value note facts; precision on that kind is 0.21 against 23 false positives.	F1 of 0.533 against a 0.70 gate, so the memory is noisier than it should be.	Narrow the schema. Restricted to tasks and commitments the same run already reaches F1 0.68.
Interactive latency is roughly ten times the target	Memory bandwidth on an integrated GPU, not an algorithmic fault. The intended discrete GPU was never available.	About 25 s to an answer. Usable for background work, not for conversation.	Re-measure on a discrete GPU before concluding anything; speculative decoding trades arithmetic for memory traffic.
Unseen injections still reach the task list	The guard is pattern-based and conservative; social engineering phrased as ordinary mail evades it.	12 of 39 untouched attacks stored as readable to-dos. None acted, leaked or wrote anything.	A third, unseen corpus is required before any further tuning can be measured honestly.
The zero-network claim cannot be made for the whole process tree	WebView2 opens one HTTPS connection per run despite every available mitigation; it is outside the application's code.	The honest statement is narrower than the headline one, and the project says so in its own documentation.	Identify the endpoint with an elevated capture; failing that, draw the notch natively and use WebView2 only for windows the user opens.
Five of seven backend families are unproven on hardware	Only CPU and AMD Vulkan devices were available; the rest are exercised against simulated adapter reports.	The arbitration logic is tested; the vendors' drivers are not.	A closed beta on varied hardware is the only way to close this — simulated adapters cannot.

Not listed because they have not been measured rather than because they are absent: resource use over 8 hours, 7-day stability, and the contribution of the vector retrieval channel, which is implemented but was never run end to end.

ROADMAP

Close the measurement gaps before building anything new

IMPLEMENTED TODAY

- Capture ladder, redaction and encrypted store
- Extraction with evidence verification and the injection guard
- Hybrid retrieval, Ask with per-sentence citations
- Rollups, morning brief, open loops, draft assistant
- Backend detection, selection and fallback; notch interface

NEXT — CLOSE THE GAPS (NOT YET DONE)

- Re-run extraction, Ask and drafts on a discrete GPU
- Curb low-value note facts to lift precision
- Commission a third, unseen injection corpus
- 24-hour network and canary runs; 8-hour and 7-day stability runs
- Resolve or formally document the WebView2 connection

LATER — PLANNED, NOT STARTED

- Meeting capture and local translation (specified, Stage 8)
- Closed beta on varied hardware — the only route to real backend coverage
- Signed installer, SBOM per release, opt-in signed updater
- Speculative decoding for memory-bandwidth-limited GPUs
- Cross-platform acquisition behind the existing Platform trait

The ordering is the point. Stages 8 to 10 were gated behind the release criteria before any of them were measured, and that gate has not been met. Building meeting capture on top of an extraction pipeline that scores 0.533 would add surface area to something not yet known to work.

CONCLUSION

Put the guarantees where they can be tested

- **The problem**
  An assistant is useful because it can see your screen, which is exactly why you would not want it sent away.

- **The approach**
  Run the model locally, give it no tools, and let deterministic code verify everything it proposes before anything is stored or shown.

- **The contribution**
  An architecture in which privacy and provenance are enforced by code, and a measurement of which of its guarantees survived a real 4B model on commodity hardware.

- **The finding**
  Every criterion enforced by code was met. Every criterion depending on the model, the GPU or a third-party runtime was not. That split is the most portable result here.

RESOURCES

Tessera repository github.com/webex042/Tessera
Source, README, decision records, stage verification reports and the evaluation results quoted throughout this deck.

llama.cpp github.com/ggml-org/llama.cpp
The inference engine, pinned at build b11900.

Qwen3-VL technical report arxiv.org/abs/2511.21631
The vision-language model family used here.

Indirect prompt injection arxiv.org/abs/2302.12173
Greshake et al., AISec 2023 — the attack slide 9 defends against.

OWASP Top 10 for LLM Applications owasp.org
Prompt injection as the first entry; the framing used for the threat model.

Full thesis — "Tessera: A Privacy-Preserving, Hardware-Independent Local Inference and Screen-Aware Memory System for Windows" — is in the repository under thesis/, with the evaluation data behind every number in this deck.